

Neo4j Cypher

25th May 2025

R00225669

Rebecca Lordan

Neo4J CRUD Movie Database

Task 1

Movie Graph Nodes

Movie Node, label is Movie and properties are title, released (year), tagline

Person Node, label is Person and properties are name, born (year)

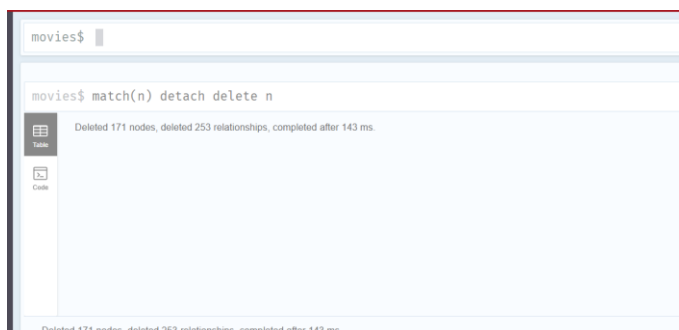
Movie Graph Relationships

ACTED_IN, connects a Person to a Movie

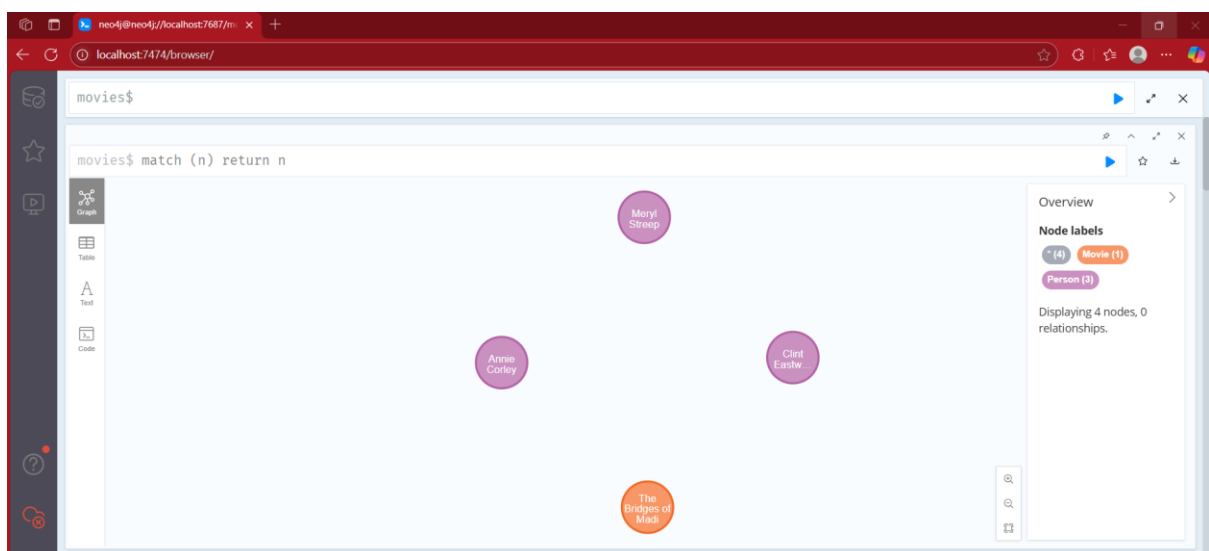
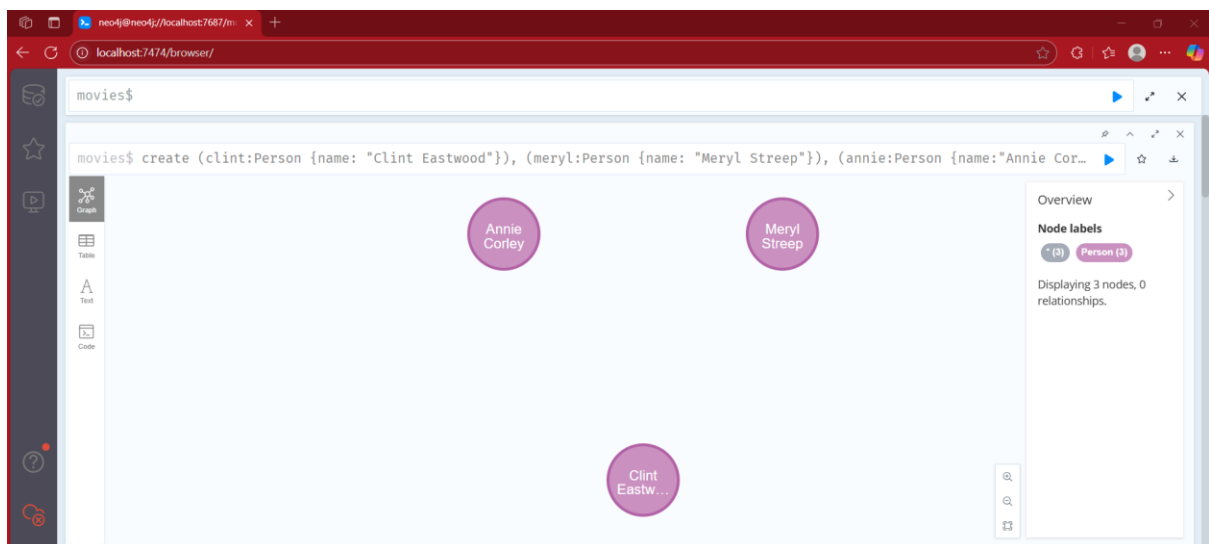
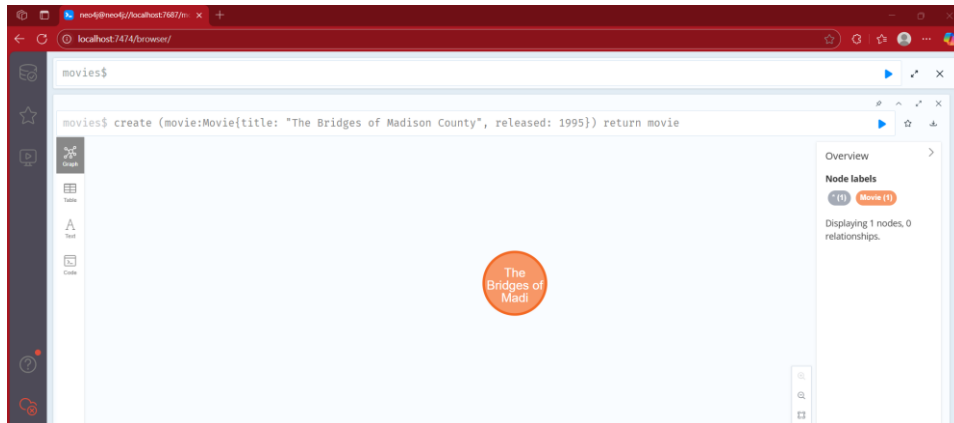
DIRECTED, connects a Person to a Movie

PRODUCED, connects a Person to a Movie

WROTE, connects a Person to a Movie



1. Add movie, actor (three main characters), director as vertices and ACTED_IN/DIRECTED edges for movie “The Bridges of Madison County” The Bridges of Madison County (1995) - IMDb [PS. Do not insert vertices that already exist in the database, and make sure the database does not contain any dummy nodes]



movies\$

```
movies$ match (clint:Person {name: "Clint Eastwood"}), (movie:Movie {title: "The Bridges of Madison County"}) create (clint)-
```

```
graph TD; ClintEastw((Clint Eastw...)) -- ACTED_IN --> BridgesMadi((The Bridges of Madi))
```

Overview

Node labels

- (2) Person (1) Movie (1)

Relationship types

- (1) ACTED_IN (1)

Displaying 2 nodes, 0 relationships.

Use Ctrl or Shift + scroll to zoom
Don't show again

movies\$

```
movies$ match (annie:Person {name: "Annie Corley"}), (movie:Movie {title: "The Bridges of Madison County"}) create (annie)-[acted_in:ACTED_IN]-(movie) return annie, acted_in, movie
```

```
graph TD; AnnieCorley((Annie Corley)) -- ACTED_IN --> BridgesMadi((The Bridges of Madi))
```

movies\$

```
movies$ match (n) return n
```

```
graph TD; AnnieCorley((Annie Corley)) -- ACTED_IN --> BridgesMadi((The Bridges of Madi)); MerylStreep((Meryl Streep)) -- ACTED_IN --> BridgesMadi; ClintEastw((Clint Eastw...)) -- ACTED_IN --> BridgesMadi
```


2. Find all actors that directed a movie they also acted in and return actor and movies



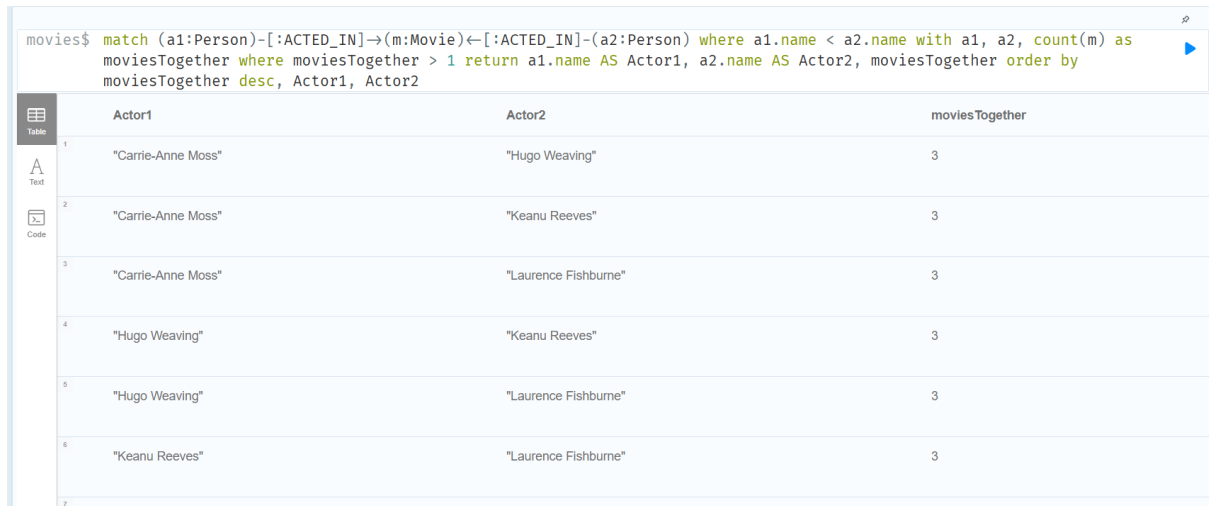
The screenshot shows a Neo4j browser window with a Cypher query executed. The query finds actors who have both acted in and directed a movie. The results are displayed in a table with two columns: Actor and Movie.

```
movies$ match (actor:Person)-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(actor) return actor.name as Actor, movie.title as Movie order by actor.name, movie.title
```

	Actor	Movie
1	"Clint Eastwood"	"The Bridges of Madison County"
2	"Clint Eastwood"	"Unforgiven"
3	"Danny DeVito"	"Hoffa"
4	"Tom Hanks"	"That Thing You Do"

Started streaming 4 records after 19 ms and completed after 33 ms.

3. Find all actor pairs that acted in multiple movies together



The screenshot shows a Neo4j browser window with a Cypher query executed. The query finds pairs of actors who have acted in multiple movies together. The results are displayed in a table with three columns: Actor1, Actor2, and moviesTogether.

```
movies$ match (a1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(a2:Person) where a1.name < a2.name with a1, a2, count(m) as moviesTogether where moviesTogether > 1 return a1.name AS Actor1, a2.name AS Actor2, moviesTogether order by moviesTogether desc, Actor1, Actor2
```

	Actor1	Actor2	moviesTogether
1	"Carrie-Anne Moss"	"Hugo Weaving"	3
2	"Carrie-Anne Moss"	"Keanu Reeves"	3
3	"Carrie-Anne Moss"	"Laurence Fishburne"	3
4	"Hugo Weaving"	"Keanu Reeves"	3
5	"Hugo Weaving"	"Laurence Fishburne"	3
6	"Keanu Reeves"	"Laurence Fishburne"	3
7			

4. Find all movies that Keanu Reeves has acted in and the co-actors that acted in the same movie, returning the movie title and the list of co-actors that Keanu Reeves worked with.

The screenshot shows the Neo4j Browser interface. The Cypher query entered is:

```
movies$ match (keanu:Person {name: "Keanu Reeves"})-[:ACTED_IN]-(movie:Movie)-[:ACTED_IN]-(coActor:Person) where coActor.name <> "Keanu Reeves" return movie.title AS Movie, collect(DISTINCT coActor.name) as CoActors order by movie.title
```

The results are displayed in a table with two columns: Movie and CoActors.

	Movie	CoActors
1	"Johnny Mnemonic"	["Dina Meyer", "Ice-T", "Takeshi Kitano"]
2	"Something's Gotta Give"	["Diane Keaton", "Jack Nicholson"]
3	"The Devil's Advocate"	["Charlize Theron", "Al Pacino"]
4	"The Matrix"	["Emil Eifrem", "Hugo Weaving", "Laurence Fishburne", "Carrie-Anne Moss"]
5	"The Matrix Reloaded"	["Carrie-Anne Moss", "Laurence Fishburne", "Hugo Weaving"]
6	"The Matrix Revolutions"	["Laurence Fishburne", "Carrie-Anne Moss", "Hugo Weaving"]

5. Delete the movies that have the lowest rating.

The screenshot shows the Neo4j Browser interface. The Cypher query entered is:

```
movies$ match ()-[rating:RATED]->() return min(rating.rating) AS LowestRating
```

The results are displayed in a table with one column: LowestRating.

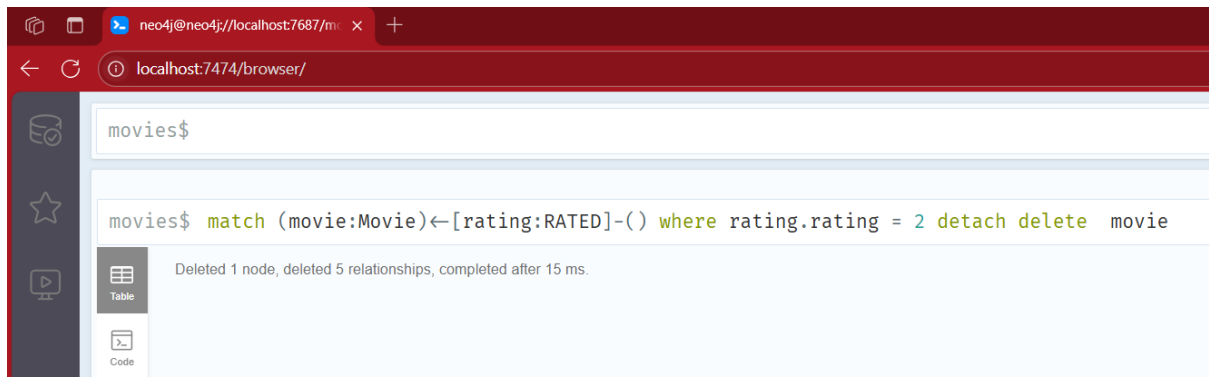
	LowestRating
1	2

The screenshot shows the Neo4j Browser interface. The Cypher query entered is:

```
movies$ match (movie:Movie)-[:RATED]-() where rating.rating = 2 return distinct movie.title
```

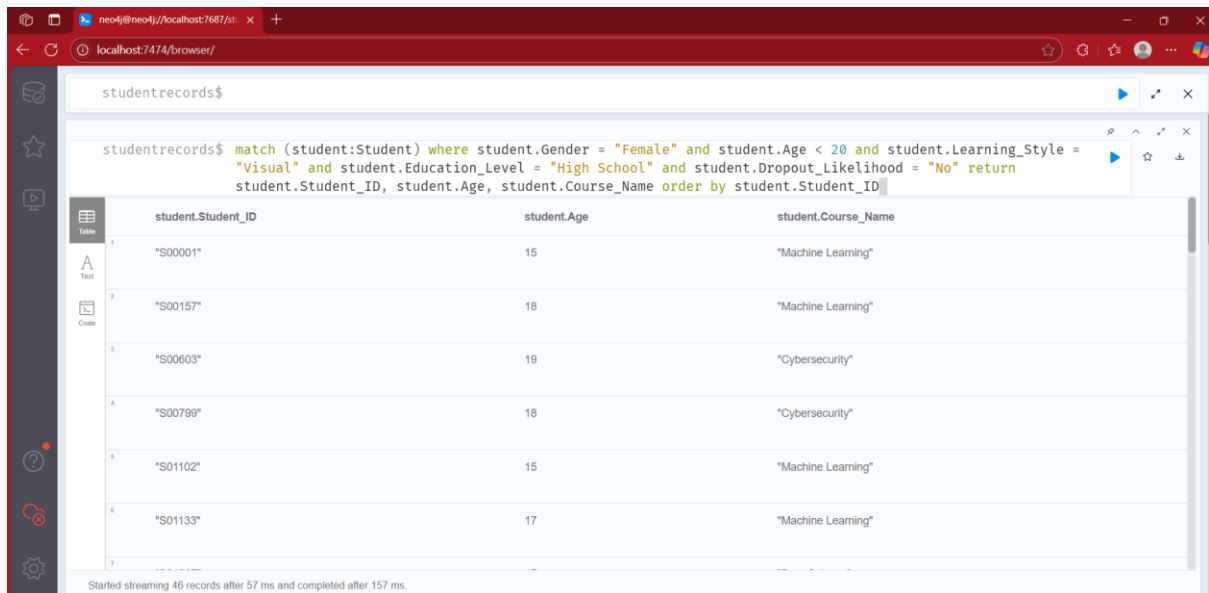
The results are displayed in a table with one column: movie.title.

	movie.title
1	"The Bridges of Madison County"

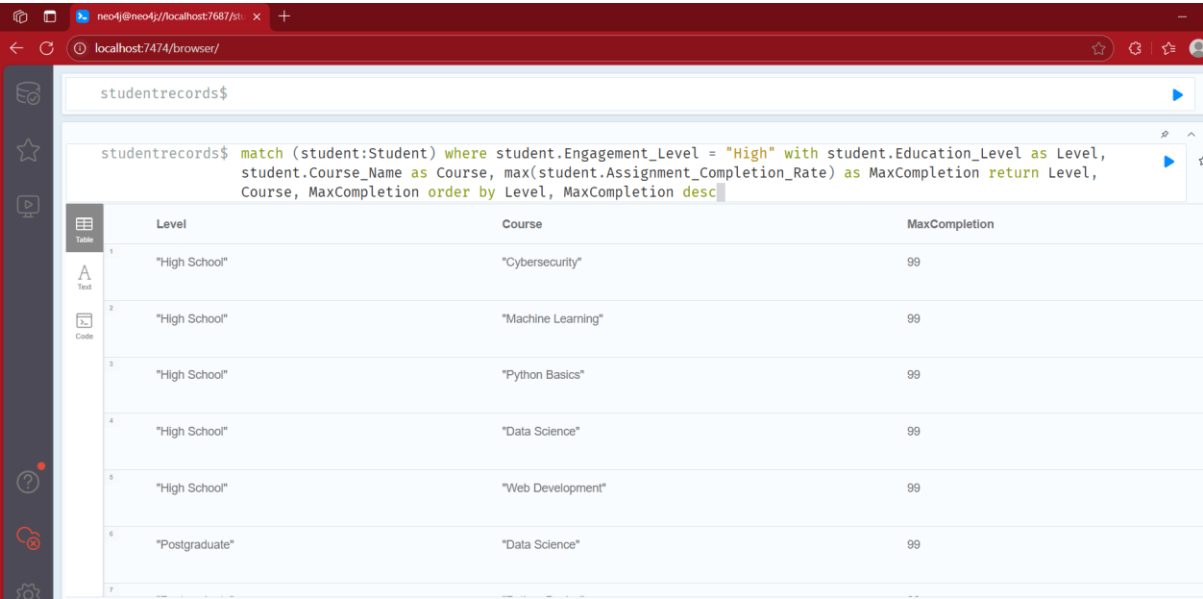


Task 2

1. List all female young female students, age below 20, prefer to visual learning style on their courses in high school and they are unlikely to drop out their courses.



2. List all courses at various educational level that have been actively engaged by students and have highest assignments completion rate.



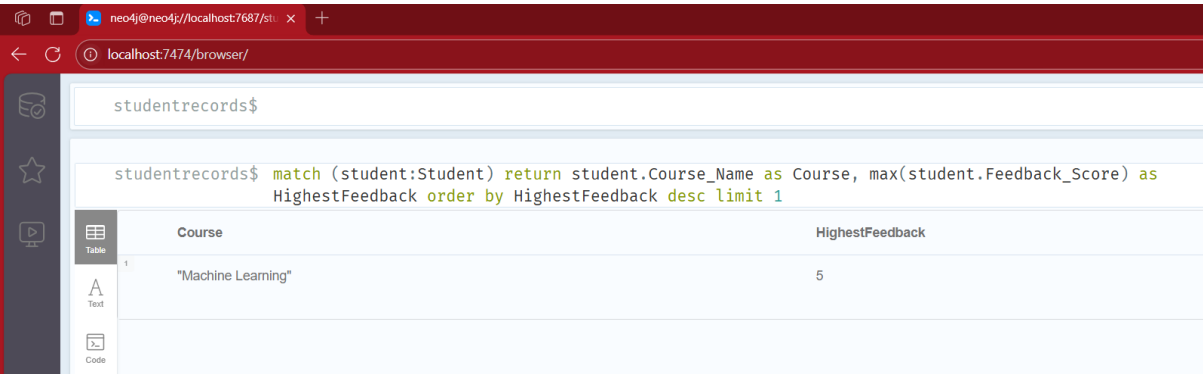
The screenshot shows the Neo4j Browser interface. The query editor contains the following Cypher query:

```
studentrecords$ match (student:Student) where student.Engagement_Level = "High" with student.Education_Level as Level, student.Course_Name as Course, max(student.Assignment_Completion_Rate) as MaxCompletion return Level, Course, MaxCompletion order by Level, MaxCompletion desc
```

The results are displayed in a table with the following data:

	Level	Course	MaxCompletion
1	"High School"	"Cybersecurity"	99
2	"High School"	"Machine Learning"	99
3	"High School"	"Python Basics"	99
4	"High School"	"Data Science"	99
5	"High School"	"Web Development"	99
6	"Postgraduate"	"Data Science"	99

3. Which course has received the highest feedback rating score?



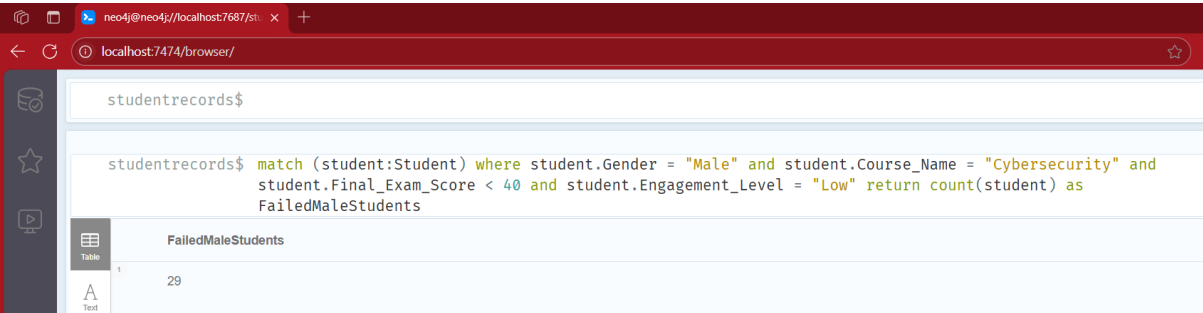
The screenshot shows the Neo4j Browser interface. The query editor contains the following Cypher query:

```
studentrecords$ match (student:Student) return student.Course_Name as Course, max(student.Feedback_Score) as HighestFeedback order by HighestFeedback desc limit 1
```

The results are displayed in a table with the following data:

	Course	HighestFeedback
1	"Machine Learning"	5

4. How many male students have failed the cybersecurity final exam (below 40) due to the low engagement?



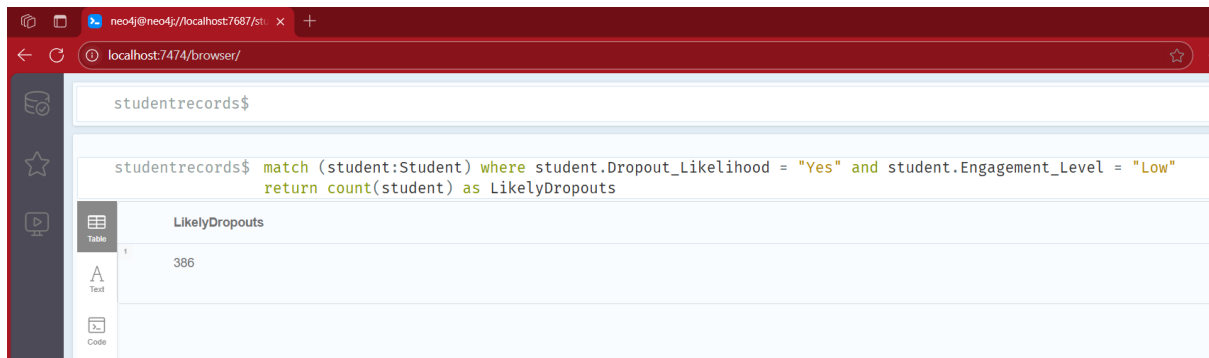
The screenshot shows the Neo4j Browser interface. The query editor contains the following Cypher query:

```
studentrecords$ match (student:Student) where student.Gender = "Male" and student.Course_Name = "Cybersecurity" and student.Final_Exam_Score < 40 and student.Engagement_Level = "Low" return count(student) as FailedMaleStudents
```

The results are displayed in a table with the following data:

	FailedMaleStudents
1	29

5. How many students will likely to drop out their courses due to low engagement and list out their learning style and their educational level.



The screenshot shows a web browser window with the address bar at `localhost:7474/browser/`. The Neo4j browser interface is visible, with a query editor on the left and a results pane on the right. The query entered is:

```
studentrecords$ match (student:Student) where student.Dropout_Likelihood = "Yes" and student.Engagement_Level = "Low"
return count(student) as LikelyDropouts
```

The results pane displays a single table with the following data:

LikelyDropouts
386